BLOCKSEC

# Security Audit Report for Bridge Contract

**Date:** April 1, 2026 **Version:** 1.0

**Contact:** contact@blocksec.com

# Contents

**Chapter 1 Introduction** **1**
1.1 About the Audit Target . . . 1
1.2 Disclaimer . . . 1
1.3 Procedure of Auditing . . . 2
1.3.1 Security Issues . . . 2
1.3.2 Additional Recommendation . . . 3
1.4 Security Model . . . 3

**Chapter 2 Findings** **4**
2.1 Security Issue . . . 4
2.1.1 Potential fund lock due to lack of check in function `remove_token()` . . . 4
2.1.2 Lack of `DefaultAccountState` extension check . . . 5
2.1.3 Lack of minimum deposit amount validation . . . 6
2.2 Recommendation . . . 7
2.2.1 Reject empty `withdrawals` in function `batch_withdraw()` . . . 7
2.2.2 Improper check in function `batch_withdraw()` . . . 13
2.3 Note . . . 14
2.3.1 Potential centralization risks . . . 14
2.3.2 Ensure the off-chain event monitoring matches on-chain event definitions 14

## Report Manifest

| Item | Description |
|---|---|
| Client | Pacifica |
| Target | Bridge Contract |

## Version History

| Version | Date | Description |
|---|---|---|
| 1.0 | April 1, 2026 | First release |

## Signature

Digitally signed by: BlockSec (https://blocksec.com)

SHA1 digest of public key: BAB106D3708BC6DE0963CE05E9478C24B326AE1C

Date: 2026-04-07 07:50:26+0000

**About BlockSec** BlockSec focuses on the security of the blockchain ecosystem and collaborates with leading DeFi projects to secure their products. BlockSec is founded by top-notch security researchers and experienced experts from both academia and industry. They have published multiple blockchain security papers in prestigious conferences, reported several zero-day attacks of DeFi applications, and successfully protected digital assets that are worth more than 14 million dollars by blocking multiple attacks. They can be reached at Email, Twitter and Medium.

# Chapter 1 Introduction

## 1.1 About the Audit Target

| Information | Description |
|---|---|
| Type | Smart Contract |
| Language | Rust |
| Approach | Semi-automatic and manual verification |

The audit target (hereinafter referred to as the Target) of this audit is the code repository [1] of Bridge Contract of Pacifica.

Bridge Contract is a cross-chain bridge protocol on Solana designed to facilitate transfers of SOL and SPL tokens across different chains. The protocol operates by emitting on-chain logs that are observed and processed by an off-chain relayer service, which coordinates and executes corresponding actions on the destination chain. The system is composed of two dedicated programs: sol-bridge, which handles native SOL bridging, and spl-bridge, which manages SPL token registration and transfers. This separation allows for clearer responsibility boundaries and more flexible handling of different asset types within the bridging workflow.

Note this audit only focuses on the smart contracts in the following directories/files:

- programs/sol-bridge
- programs/spl-bridge

Other files are not within the scope of the audit. Additionally, all dependencies of the Target are considered reliable in terms of both functionality and security, and are therefore not included in the audit scope.

The auditing process is iterative. Specifically, we would audit the commits that fix the discovered issues. If there are new issues, we will continue this process. The commit SHA values during the audit are shown in the following table. Our audit report is responsible for the code in the initial version (`Version 1`), as well as new code (in the following versions) to fix issues in the audit report. Code prior to and including the baseline version (`Version 0`), where applicable, is outside the scope of this audit and assumes to be reliable and secure.

| Project | Version | Commit Hash |
|---|---|---|
| Bridge Contract | `Version 1` | `15d0aa7a4269beba47f67f9af35eb7d85525adaf` |
| | `Version 1` | `4dd0ff83094a5cfb61856444953356d50e62ee40` |

## 1.2 Disclaimer

This audit report does not constitute investment advice or a personal recommendation. It does not consider, and should not be interpreted as considering or having any bearing on, the potential economics of a token, token sale or any other product, service or other asset.

[1] https://github.com/pacifica-fi/bridge-contract

Any entity should not rely on this report in any way, including for the purpose of making any decisions to buy or sell any token, product, service or other asset.

This audit report is not an endorsement of any particular project or team, and the report does not guarantee the security of any particular project. This audit does not give any warranties on discovering all security issues of the Target, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit cannot be considered comprehensive, we always recommend proceeding with independent audits and a public bug bounty program to ensure the security of the Target.

The scope of this audit is limited to the code mentioned in Section 1.1. Unless explicitly specified, the security of the language itself (e.g., the solidity language), the underlying compiling toolchain and the computing infrastructure are out of the scope.

## 1.3 Procedure of Auditing

We perform the audit according to the following procedure.

- **Vulnerability Detection** We first scan the Target with automatic code analyzers, and then manually verify (reject or confirm) the issues reported by them.
- **Semantic Analysis** We study the business logic of the Target and conduct further investigation on the possible vulnerabilities using an automatic fuzzing tool (developed by our research team). We also manually analyze possible attack scenarios with independent auditors to cross-check the result.
- **Recommendation** We provide some useful advice to developers from the perspective of good programming practice, including gas optimization, code style, and etc.

We show the main concrete checkpoints in the following.

### 1.3.1 Security Issues

* Access control
* Permission management
* Whitelist and blacklist mechanisms
* Initialization consistency
* Improper use of the proxy system
* Reentrancy
* Denial of Service (DoS)
* Untrusted external call and control flow
* Exception handling
* Data handling and flow
* Events operation
* Error-prone randomness
* Oracle security
* Business logic correctness
* Semantic and functional consistency
* Emergency mechanism

* Economic and incentive impact

### 1.3.2 Additional Recommendation

* Gas optimization
* Code quality and style

**Note** *The previous checkpoints are the main ones. We may use more checkpoints during the auditing process according to the functionality of the project.*

## 1.4 Security Model

To evaluate the risk, we follow the standards or suggestions that are widely adopted by both industry and academy, including OWASP Risk Rating Methodology [2] and Common Weakness Enumeration [3]. The overall *severity* of the risk is determined by *likelihood* and *impact*. Specifically, likelihood is used to estimate how likely a particular vulnerability can be uncovered and exploited by an attacker, while impact is used to measure the consequences of a successful exploit.

In this report, both likelihood and impact are categorized into two ratings, i.e., *high* and *low* respectively, and their combinations are shown in Table 1.1.

**Table 1.1:** Vulnerability Severity Classification

| Impact | Likelihood: *High* | Likelihood: *Low* |
|---|---|---|
| *High* | High | Medium |
| *Low* | Medium | Low |

Accordingly, the severity measured in this report are classified into three categories: **High**, **Medium**, **Low**. For the sake of completeness, **Undetermined** is also used to cover circumstances when the risk cannot be well determined.

Furthermore, the status of a discovered item will fall into one of the following five categories:

- **Undetermined** No response yet.
- **Acknowledged** The item has been received by the client, but not confirmed yet.
- **Confirmed** The item has been recognized by the client, but not fixed yet.
- **Partially Fixed** The item has been confirmed and partially fixed by the client.
- **Fixed** The item has been confirmed and fixed by the client.

[2] https://owasp.org/www-community/OWASP_Risk_Rating_Methodology

[3] https://cwe.mitre.org/

# Chapter 2 Findings

In total, we found **three** potential security issues. Besides, we have **two** recommendations and **two** notes.

- Medium Risk: 1
- Low Risk: 2
- Recommendation: 2
- Note: 2

| ID | Severity | Description | Category | Status |
|---|---|---|---|---|
| 1 | Medium | Potential fund lock due to lack of check in function `remove_token()` | Security Issue | Fixed |
| 2 | Low | Lack of `DefaultAccountState` extension check | Security Issue | Fixed |
| 3 | Low | Lack of minimum deposit amount validation | Security Issue | Confirmed |
| 4 | - | Reject empty `withdrawals` in function `batch_withdraw()` | Recommendation | Fixed |
| 5 | - | Improper check in function `batch_withdraw()` | Recommendation | Fixed |
| 6 | - | Potential centralization risks | Note | - |
| 7 | - | Ensure the off-chain event monitoring matches on-chain event definitions | Note | - |

The details are provided in the following sections.

## 2.1 Security Issue

### 2.1.1 Potential fund lock due to lack of check in function `remove_token()`

**Severity** Medium

**Status** Fixed in `Version 2`

**Introduced by** `Version 1`

**Description** The function `remove_token()` removes a supported token by setting `supported_token.is_active = false`. The function `batch_withdraw()` can only withdraw tokens that are currently supported. If a token is removed while assets still remain in `bridge_token_account`, function `batch_withdraw()` can no longer be used to withdraw them. Since the protocol does not support resetting `supported_token.is_active` to true, the assets will be permanently locked in `bridge_token_account`.

```
impl RemoveToken<'_> {
    pub fn handle(ctx: Context<RemoveToken>) -> Result<()> {
        let clock = Clock::get()?;

        let supported_token = &mut ctx.accounts.supported_token;
        supported_token.is_active = false;

```

```
        emit_cpi!(TokenRemoved {
            mint: ctx.accounts.mint.key(),
            timestamp: clock.unix_timestamp,
        });

        Ok(())
    }
}
```

**Listing 2.1:** programs/spl-bridge/src/instructions/remove_token.rs

**Impact** Any tokens that remain in `bridge_token_account` after `remove_token` is invoked are permanently irrecoverable absent a program upgrade.

**Suggestion** Revise the logic accordingly.

### 2.1.2 Lack of `DefaultAccountState` extension check

**Severity** Low

**Status** Fixed in `Version 2`

**Introduced by** `Version 1`

**Description** The protocol allows privileged accounts to register supported tokens via function `add_token()`. However, the function `add_token()` does not check for certain Token-2022 extensions, such as `DefaultAccountState`.

If a mint is configured with `DefaultAccountState::Frozen`, Anchor's `init` will initialize `bridge_token_account` and `fallback_token_account` in a frozen state, they will be unable to receive tokens.

```
impl AddToken<'_> {
    pub fn handle(ctx: Context<AddToken>) -> Result<()> {
        // Check for dangerous Token-2022 extensions
        if ctx.accounts.token_program.key() == spl_token_2022::ID {
            let mint_account_info = ctx.accounts.mint.to_account_info();
            let mint_data = mint_account_info.try_borrow_data()?;
            let mint_state = StateWithExtensions::<Mint2022>::unpack(&mint_data)
                .map_err(|_| BridgeError::UnsupportedTokenExtension)?;

            // Reject tokens with dangerous extensions
            require!(
                mint_state.get_extension::<NonTransferable>().is_err(),
                BridgeError::UnsupportedTokenExtension
            );
            require!(
                mint_state.get_extension::<PermanentDelegate>().is_err(),
                BridgeError::UnsupportedTokenExtension
            );
            require!(
                mint_state
                    .get_extension::<ConfidentialTransferMint>()
                    .is_err(),
                BridgeError::UnsupportedTokenExtension
```

```
            );
            require!(
                mint_state.get_extension::<TransferHook>().is_err(),
                BridgeError::UnsupportedTokenExtension
            );
            // Transfer fees would cause accounting issues - deposited amount != withdrawn
                amount
            require!(
                mint_state.get_extension::<TransferFeeConfig>().is_err(),
                BridgeError::UnsupportedTokenExtension
            );
        }
    }
}
```

**Listing 2.2:** programs/spl-bridge/src/instructions/add_token.rs

**Impact** The frozen `bridge_token_account` and `fallback_token_account` will not be able to receive tokens.

**Suggestion** Add the `DefaultAccountState` extension check.

### 2.1.3 Lack of minimum deposit amount validation

**Severity** Low

**Status** Confirmed

**Introduced by** `Version 1`

**Description** The program `sol-bridge` facilitates native SOL transfers from users to the bridge vault through the function `handle()`. The function `handle()` does not implement a minimum deposit check that synchronizes with the off-chain processing logic. This inconsistency causes the state manager to consume the deposit nonce without crediting funds to the user, which leads to the irreversible loss of small-value deposits.

The program `spl-bridge` has a similar issue.

```
impl DepositSol<'_> {
    pub fn handle(ctx: Context<DepositSol>, recipient: Pubkey, amount: u64) -> Result<()> {
        require!(amount > 0, BridgeError::ZeroAmount);
        require!(
            recipient != Pubkey::default(),
            BridgeError::InvalidRecipient
        );

        let clock = Clock::get()?;

        system_program::transfer(
            CpiContext::new(
                ctx.accounts.system_program.to_account_info(),
                system_program::Transfer {
                    from: ctx.accounts.depositor.to_account_info(),
                    to: ctx.accounts.bridge_authority.to_account_info(),
                },
```

```
            ),
            amount,
        )?;

        let deposit_nonce = ctx.accounts.bridge_config.deposit_nonce;

        emit_cpi!(Deposit {
            source: ctx.accounts.depositor.key(),
            recipient,
            mint: Pubkey::default(),
            amount,
            deposit_nonce,
            timestamp: clock.unix_timestamp,
        });

        ctx.accounts.bridge_config.deposit_nonce =
            deposit_nonce.checked_add(1).ok_or(BridgeError::Overflow)?;

        Ok(())
    }
}
```

**Listing 2.3:** programs/sol-bridge/src/instructions/deposit_sol.rs

**Impact** Off-chain systems will consume deposit nonces without updating account balances, which may leads to the irreversible loss of small-value deposits.

**Suggestion** Revise the logic accordingly.

**Feedback from the project** The team confirmed that irreversible loss of small-value deposits is an intentional design choice. Minimum deposit amounts adjust based on asset prices. For simplification, this dynamic logic and enforcement of minimum deposit amounts is only tracked on the backend. Minimum deposit amounts will be clearly documented so users are aware of this irreversible loss behavior.

## 2.2 Recommendation

### 2.2.1 Reject empty `withdrawals` in function `batch_withdraw()`

**Status** Fixed in `Version 2`

**Introduced by** `Version 1`

**Description** In the programs `spl-bridge` and `sol-bridge`, the function `batch_withdraw()` releases bridged assets by iterating over the input `withdrawals` array and transferring funds to each recipient. However, the function does not verify that the input `withdrawals` array is non-empty. If an empty array is provided, no transfer is executed; nevertheless, the function still emits the event `BatchWithdrawalCompleted` and increments the variable `withdraw_nonce`, which is operationally meaningless and may cause unnecessary nonce consumption and misleading off-chain accounting.

```
pub fn batch_withdraw<'info>(
```

```
    ctx: Context<'_, '_, 'info, 'info, BatchWithdraw<'info>>,
    nonce: u64,
    withdrawals: Vec<WithdrawalRequest>,
) -> Result<()> {
    BatchWithdraw::handle(ctx, nonce, withdrawals)
}
```

**Listing 2.4:** programs/sol-bridge/src/lib.rs

```
pub fn handle(
    ctx: Context<'_, '_, 'info, 'info, BatchWithdraw<'info>>,
    nonce: u64,
    withdrawals: Vec<WithdrawalRequest>,
) -> Result<()> {
    require!(
        nonce == ctx.accounts.bridge_config.withdraw_nonce,
        BridgeError::InvalidNonce
    );

    require!(
        ctx.remaining_accounts.len() == withdrawals.len(),
        BridgeError::Unauthorized
    );

    let clock = Clock::get()?;
    let bridge_authority_bump = ctx.bumps.bridge_authority;
    let authority_seeds: &[&[u8]] = &[BRIDGE_AUTHORITY, &[bridge_authority_bump]];

    let mut successful_withdrawals: Vec<WithdrawalRecord> =
        Vec::with_capacity(withdrawals.len());

    for (idx, withdrawal) in withdrawals.iter().enumerate() {
        require!(withdrawal.amount > 0, BridgeError::ZeroAmount);
        require!(
            withdrawal.recipient != Pubkey::default(),
            BridgeError::InvalidRecipient
        );

        let recipient_info = &ctx.remaining_accounts[idx];

        require!(
            recipient_info.key() == withdrawal.recipient,
            BridgeError::InvalidRecipient
        );

        let bridge_balance = ctx.accounts.bridge_authority.lamports();
        require!(
            bridge_balance >= withdrawal.amount,
            BridgeError::InsufficientBridgeBalance
        );

        let recipient_lamports = recipient_info.lamports();

```

```
        if recipient_lamports == 0 {
            system_program::transfer(
                CpiContext::new_with_signer(
                    ctx.accounts.system_program.to_account_info(),
                    system_program::Transfer {
                        from: ctx.accounts.bridge_authority.to_account_info(),
                        to: ctx.accounts.fallback_authority.to_account_info(),
                    },
                    &[authority_seeds],
                ),
                withdrawal.amount,
            )?;

            emit_cpi!(WithdrawalToFallback {
                recipient: withdrawal.recipient,
                mint: Pubkey::default(),
                amount: withdrawal.amount,
                reason: "Recipient account does not exist (0 lamports)".to_string(),
                timestamp: clock.unix_timestamp,
            });
        } else {
            let transfer_result = system_program::transfer(
                CpiContext::new_with_signer(
                    ctx.accounts.system_program.to_account_info(),
                    system_program::Transfer {
                        from: ctx.accounts.bridge_authority.to_account_info(),
                        to: recipient_info.clone(),
                    },
                    &[authority_seeds],
                ),
                withdrawal.amount,
            );

            match transfer_result {
                Ok(_) => {
                    successful_withdrawals.push(WithdrawalRecord {
                        recipient: withdrawal.recipient,
                        amount: withdrawal.amount,
                    });
                }
                Err(e) => {
                    system_program::transfer(
                        CpiContext::new_with_signer(
                            ctx.accounts.system_program.to_account_info(),
                            system_program::Transfer {
                                from: ctx.accounts.bridge_authority.to_account_info(),
                                to: ctx.accounts.fallback_authority.to_account_info(),
                            },
                            &[authority_seeds],
                        ),
                        withdrawal.amount,
                    )?;

```

```
                    emit_cpi!(WithdrawalToFallback {
                        recipient: withdrawal.recipient,
                        mint: Pubkey::default(),
                        amount: withdrawal.amount,
                        reason: format!("Transfer failed: {}", e),
                        timestamp: clock.unix_timestamp,
                    });
                }
            }
        }
    }

    emit_cpi!(BatchWithdrawalCompleted {
        nonce,
        mint: Pubkey::default(),
        withdrawer: ctx.accounts.withdrawer.key(),
        withdrawals: successful_withdrawals,
        timestamp: clock.unix_timestamp,
    });

    ctx.accounts.bridge_config.withdraw_nonce =
        nonce.checked_add(1).ok_or(BridgeError::Overflow)?;

    Ok(())
}
```

**Listing 2.5:** programs/sol-bridge/src/instructions/batch_withdraw.rs

```
pub fn batch_withdraw<'info>(
    ctx: Context<'_, '_, 'info, 'info, BatchWithdraw<'info>>,
    mint: Pubkey,
    nonce: u64,
    withdrawals: Vec<WithdrawalRequest>,
) -> Result<()> {
    BatchWithdraw::handle(ctx, mint, nonce, withdrawals)
}
```

**Listing 2.6:** programs/spl-bridge/src/lib.rs

```
pub fn handle(
    ctx: Context<'_, '_, 'info, 'info, BatchWithdraw<'info>>,
    mint: Pubkey,
    nonce: u64,
    withdrawals: Vec<WithdrawalRequest>,
) -> Result<()> {
    // Validate nonce matches expected value
    require!(
        nonce == ctx.accounts.supported_token.withdraw_nonce,
        BridgeError::InvalidNonce
    );

    // Validate remaining accounts length
    require!(
```

```
        ctx.remaining_accounts.len() == withdrawals.len(),
        BridgeError::Unauthorized
    );

    let clock = Clock::get()?;
    let bridge_authority_bump = ctx.bumps.bridge_authority;
    let authority_seeds: &[&[u8]] = &[BRIDGE_AUTHORITY, &[bridge_authority_bump]];

    let decimals = ctx.accounts.mint_account.decimals;

    // Track successful withdrawals for batch event
    let mut successful_withdrawals: Vec<WithdrawalRecord> =
        Vec::with_capacity(withdrawals.len());

    // Process each withdrawal
    for (idx, withdrawal) in withdrawals.iter().enumerate() {
        require!(withdrawal.amount > 0, BridgeError::ZeroAmount);
        require!(
            withdrawal.recipient != Pubkey::default(),
            BridgeError::InvalidRecipient
        );

        // Get recipient token account from remaining_accounts
        let recipient_token_info = &ctx.remaining_accounts[idx];

        // Calculate expected ATA on-chain (supports both legacy SPL and Token-2022)
        let expected_ata =
            anchor_spl::associated_token::get_associated_token_address_with_program_id(
                &withdrawal.recipient,
                &mint,
                &ctx.accounts.token_program.key(),
            );

        require!(
            recipient_token_info.key() == expected_ata,
            BridgeError::InvalidRecipient
        );

        // Check if ATA is initialized (account has data)
        let should_use_fallback = if recipient_token_info.data_is_empty() {
            Some("ATA not initialized")
        } else {
            // Parse the token account to check for frozen state and CPI guard
            let account_data = recipient_token_info.try_borrow_data()?;

            // Check for Token-2022 specific extensions
            if ctx.accounts.token_program.key() == spl_token_2022::ID {
                if let Ok(account_state) =
                    StateWithExtensions::<Account2022>::unpack(&account_data)
                {
                    // Check if account is frozen
                    if account_state.base.is_frozen() {
                        Some("Account frozen")
```

```
                    }
                    // Check if CPI Guard is enabled
                    else if account_state
                        .get_extension::<CpiGuard>()
                        .map(|g| g.lock_cpi.into())
                        .unwrap_or(false)
                    {
                        Some("CPI Guard enabled")
                    } else {
                        None
                    }
                } else {
                    Some("Failed to parse account")
                }
            } else {
                // Legacy SPL token - just check frozen state
                // TokenAccount is 165 bytes, frozen flag is at offset 108
                if account_data.len() >= 109 && account_data[108] == 2 {
                    // AccountState::Frozen = 2
                    Some("Account frozen")
                } else {
                    None
                }
            }
        };

        if let Some(reason) = should_use_fallback {
            // Transfer to fallback
            transfer_checked(
                CpiContext::new_with_signer(
                    ctx.accounts.token_program.to_account_info(),
                    TransferChecked {
                        from: ctx.accounts.bridge_token_account.to_account_info(),
                        mint: ctx.accounts.mint_account.to_account_info(),
                        to: ctx.accounts.fallback_token_account.to_account_info(),
                        authority: ctx.accounts.bridge_authority.to_account_info(),
                    },
                    &[authority_seeds],
                ),
                withdrawal.amount,
                decimals,
            )?;

            // Emit fallback event
            emit_cpi!(WithdrawalToFallback {
                recipient: withdrawal.recipient,
                mint,
                amount: withdrawal.amount,
                reason: reason.to_string(),
                timestamp: clock.unix_timestamp,
            });
        } else {
            // Transfer to recipient
```

```
            transfer_checked(
                CpiContext::new_with_signer(
                    ctx.accounts.token_program.to_account_info(),
                    TransferChecked {
                        from: ctx.accounts.bridge_token_account.to_account_info(),
                        mint: ctx.accounts.mint_account.to_account_info(),
                        to: recipient_token_info.clone(),
                        authority: ctx.accounts.bridge_authority.to_account_info(),
                    },
                    &[authority_seeds],
                ),
                withdrawal.amount,
                decimals,
            )?;

            // Track successful withdrawal
            successful_withdrawals.push(WithdrawalRecord {
                recipient: withdrawal.recipient,
                amount: withdrawal.amount,
            });
        }
    }

    // Emit batch completion event with all successful withdrawals
    emit_cpi!(BatchWithdrawalCompleted {
        nonce,
        mint,
        withdrawer: ctx.accounts.withdrawer.key(),
        withdrawals: successful_withdrawals,
        timestamp: clock.unix_timestamp,
    });

    // Increment withdraw nonce as LAST operation
    ctx.accounts.supported_token.withdraw_nonce =
        nonce.checked_add(1).ok_or(BridgeError::Overflow)?;

    Ok(())
}
```

**Listing 2.7:** programs/spl-bridge/src/instructions/batch_withdraw.rs

**Suggestion** Add a check to ensure the input `withdrawals` array is not empty.

### 2.2.2 Improper check in function `batch_withdraw()`

**Status** Fixed in `Version 2`

**Introduced by** `Version 1`

**Description** In the legacy `SPL` path, `BatchWithdraw` infers frozen state by reading `account_data[108] when len >= 109`. While this works for standard `TokenAccount` layouts, it is only a partial structural check and may lose the required error message.

```
            else {
```

```
                // Legacy SPL token - just check frozen state
                // TokenAccount is 165 bytes, frozen flag is at offset 108
                if account_data.len() >= 109 && account_data[108] == 2 {
                    // AccountState::Frozen = 2
                    Some("Account frozen")
                } else {
                    None
                }
            }
```

**Listing 2.8:** programs/spl-bridge/src/instructions/batch_withdraw.rs

**Suggestion** Use `spl_token::state::Account::unpack(&account_data)` and treat `Err(_)` as a fallback trigger.

## 2.3 Note

### 2.3.1 Potential centralization risks

**Introduced by** `Version 1`

**Description** In this project, several privileged roles (e.g., `withdrawer`) can conduct sensitive operations, which introduces potential centralization risks. For example, a `withdrawer` is able to transfer funds held by the `fallback_authority` to any arbitrary address. If the private keys of the privileged accounts are lost or maliciously exploited, it could pose a significant risk to the protocol.

### 2.3.2 Ensure the off-chain event monitoring matches on-chain event definitions

**Introduced by** `Version 1`

**Description** In this project, the cross-chain mechanism relies on off-chain listeners to detect and process on-chain events. It is important to ensure that the off-chain logic is implemented correctly and remains fully consistent with the on-chain logic, including event formats, parameter encoding, and processing semantics. Any mismatch between off-chain and on-chain behavior may lead to incorrect execution or unexpected states in the bridging workflow.

BOOST WEB3 THROUGH NEXT-GENERATION SECURITY & USABILITY INNOVATIONS